

MA_DEV // DEV.LOG

블로그 코드 완전 해설

& 학습 가이드

내가 만든 코드, 내가 이해하는 코드

Next.js 14

MongoDB

Docker

Nginx

Apache Kafka

Elasticsearch

Logstash

Kibana

NextAuth

TypeScript

OCI

Raspberry Pi

작성일: 2026-04-06

목차

1. 전체 아키텍처 개요
2. 디렉토리 구조
3. Docker Compose & 컨테이너 네트워크
 - 서비스별 역할 / 환경변수 / 볼륨 / 네트워크
4. 데이터베이스 — MongoDB
 - 연결 코드 / Mongoose 스키마 / 모델 설계
5. API 설계 — Next.js App Router
 - 포스트 CRUD / 스택 CRUD / 시스템 API
6. 인증 — NextAuth + GitHub OAuth
7. ELK + Kafka 파이프라인
 - nginx 로그 → Logstash → Kafka → Elasticsearch → Kibana
8. 시스템 모니터링 위젯
 - OCI 서버 상태 / Docker 컨테이너 / Raspberry Pi / GitHub 커밋
9. 프론트엔드 구조
 - 컴포넌트 계층 / 모달 스택 / SSE / apiClient
10. 보안 설계
11. 용어 사전
12. 학습 로드맵

1. 전체 아키텍처 개요

이 블로그는 단순한 정적 사이트가 아니라, 실제 서버에서 돌아가는 풀스택 서비스다. Oracle Cloud (OCI) 위에 Docker Compose로 여러 컨테이너가 같이 실행되고, 서로 HTTP/네트워크로 통신한다.

인터넷

▼ 포트 80/443

mablog_proxy (Nginx)	← 리버스 프록시, HTTPS 종료
- HTTPS 인증서 (Let's Encrypt)	
- /api/* → Next.js로 전달	

내부 네트워크 (mablog_network)

▼ 포트 8080

mablog_nextjs (Next.js 14)	← 웹앱 + API 서버
- 페이지 렌더링 (RSC)	
- REST API (/api/controller/...)	
- 시스템 API (/api/system/...)	

→ mablog_mongodb (MongoDB 7)
데이터 저장 (포스트, 스택, 라즈베리파이 상태)

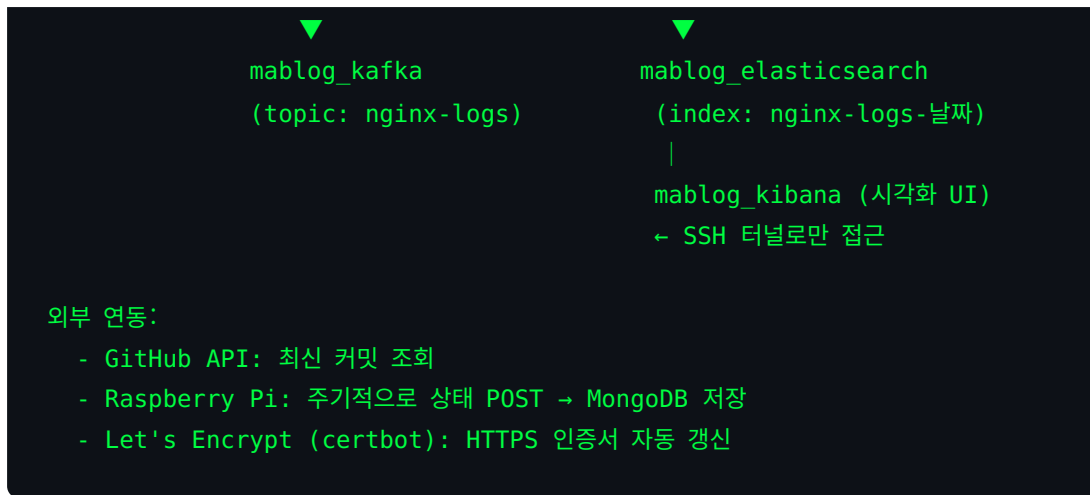
→ mablog_elasticsearch (ES 8.13)
nginx 로그 검색/저장

→ docker.sock
실행 중인 컨테이너 목록 조회

nginx 로그 파이프라인:

mablog_proxy (nginx) → access.log → 볼륨 공유

▼
mablog_logstash
(grok 파싱)



핵심 개념: 왜 Docker를 쓰나?

Docker가 없으면 서버에 Node.js, MongoDB, Nginx, Java(Kafka/ELK) 등을 직접 설치해야 한다. 버전 충돌, 설정 꼬임 등 문제가 생긴다. Docker는 각 서비스를 **격리된 컨테이너(독립 환경)**로 실행하기 때문에 서로 영향을 주지 않는다. `docker-compose.yml` 파일 하나로 전체 인프라를 정의하고, `docker-compose up -d` 한 줄로 전부 실행된다.

2. 디렉토리 구조

```
mam/
├─ src/
│  └─ app/
│     └─ api/                                ← 서버 코드 (API)
│        ├── ConnectMongoDB.ts             ← MongoDB 연결 (싱글톤)
│        ├── auth/[...nextauth]/           ← GitHub OAuth 인증
│        ├── models/                        ← MongoDB 스키마 정의
│        │   ├── posts/model_posts.ts
│        │   ├── stacks/model_stacks.ts
│        │   └─ raspberry/model_raspberry.ts
│        ├── controller/                    ← CRUD API
│        │   ├── GET/posts/route.ts
│        │   ├── GET/stack/route.ts
│        │   ├── GET/categories/route.ts
│        │   ├── INSERT/posts/route.ts
│        │   ├── INSERT/stack/route.ts
│        │   ├── DELETE/posts/route.ts
│        │   ├── DELETE/stack/route.ts
│        │   └─ UPDATE/route.ts
│        └─ system/                         ← 시스템 모니터링 API
│            ├── server/route.ts            ← OCI 서버 상태
│            ├── docker/route.ts           ← Docker 컨테이너 목록 (SSE)
│            ├── github/route.ts           ← GitHub 커밋 목록
│            ├── raspberry/route.ts        ← Raspberry Pi 상태
│            └─ nginx/route.ts             ← nginx 로그 (Elasticsearch)
│     └─ component/
│        ├── common/                       ← 재사용 컴포넌트
│        │   ├── getServerState.tsx        ← OCI 위젯
│        │   ├── getDockerState.tsx        ← Docker 위젯
│        │   ├── githubActivity.tsx        ← GitHub 커밋 위젯
│        │   ├── raspberryStatus.tsx       ← Raspberry Pi 위젯
│        │   ├── nginxLogWidget.tsx        ← nginx 로그 위젯
│        │   ├── clockWidget.tsx           ← 시계 위젯
│        │   ├── categoryFilter.tsx        ← 카테고리 필터
│        │   ├── modal.tsx                 ← 모달 컴포넌트
│        │   ├── authBtn.tsx               ← GitHub 로그인 버튼
│        │   └─ ...
│        └─ pageComponent/main/            ← 메인 페이지 전용
│            ├── mainPageComponent.tsx     ← 대시보드 진입점
│            ├── aboutMe.tsx                ← 자기소개 + 기술스택
│            └─ introModal.tsx             ← ABOUT 모달
```

```

|       |       | writeForm.tsx      ← 포스트 작성 폼
|       |       | writeFormFix.tsx  ← 포스트 수정 폼
|       |       |
|       |       | lib/
|       |       |   | apiClient.ts    ← 클라이언트용 fetch 함수 모음
|       |       |   | css/
|       |       |   |   | common/main.css    ← 대시보드 레이아웃
|       |       |   |   | common/crt.css     ← CRT 터미널 스타일
|       |       |   |   | common/modal.css   ← 모달 스타일
|       |       |
|       |       | logstash/
|       |       |   | logstash.yml          ← Logstash 기본 설정
|       |       |   | pipeline/nginx.conf   ← 파이프라인 정의
|       |       |
|       |       | nginx.conf                ← Nginx 설정
|       |       | docker-compose.yml        ← 전체 인프라 정의
|       |       | Dockerfile                ← Next.js 컨테이너 빌드 설정
|       |       | .env.production           ← 환경변수 (비밀값, git에 없음)

```

App Router란? Next.js 13부터 도입된 폴더 기반 라우팅. `src/app/api/controller/GET/posts/route.ts` 파일이 있으면 자동으로 `GET /api/controller/GET/posts` 엔드포인트가 생긴다. 파일 경로 = URL 경로.

3. Docker Compose & 컨테이너 네트워크

3-1. docker-compose.yml 전체 구조

총 8개 서비스가 `mablog_network` 라는 하나의 가상 네트워크 안에서 실행된다. 같은 네트워크 안에 있으면 컨테이너 이름으로 서로 접근 가능하다 (`mablog_mongodb:27017` 처럼).

서비스명	이미지	역할	외부 포트
db (mablog_mongodb)	mongo:7.0	데이터베이스	127.0.0.1:27017 (로컬만)
mablog_nextjs	커스텀 Dockerfile	웹앱 + API	내부 8080만
mablog_proxy	nginx:stable-alpine	리버스 프록시	80, 443
certbot	certbot/certbot	HTTPS 인증서 갱신	없음
mablog_kafka	apache/kafka:3.7.0	메시지 큐	없음 (내부만)
mablog_elasticsearch	elasticsearch:8.13.4	로그 저장/검색	없음 (내부만)
mablog_logstash	logstash:8.13.4	로그 수집/파싱	없음
mablog_kibana	kibana:8.13.4	ES 시각화 UI	127.0.0.1:5601 (로컬만)

3-2. 주요 키워드 설명

volumes (볼륨)

컨테이너는 종료하면 내부 데이터가 사라진다. 볼륨을 쓰면 **호스트 디렉토리**와 **컨테이너 디렉토리**를 **연결**해서 데이터를 영구 보존한다.

```
volumes:  
  - ./mongo_data:/data/db    # 호스트의 ./mongo_data ↔ 컨테이너의 /data/db
```

```
- ./nginx_logs:/var/log/nginx # nginx 로그를 logstash가 같이 읽을 수 있게
```

핵심: nginx 컨테이너와 logstash 컨테이너가 `./nginx_logs` 를 공유 폴더처럼 사용한다.
nginx가 access.log를 쓰면, logstash가 그 파일을 읽는다.

expose vs ports

```
expose: ['8080'] # Docker 내부 네트워크에서만 접근 가능 (외부 차단)
ports: ['80:80'] # 호스트의 80번 포트를 컨테이너의 80번 포트에 연결 (외부 접근 가능)
ports: ['127.0.0.1:27017:27017'] # 로컬호스트에서만 접근 가능 (외부 차단)
```

networks - aliases

```
mablog_elasticsearch:
  networks:
    mablog_network:
      aliases:
        - elasticsearch # 언더스코어 없는 별명
```

컨테이너 이름 `mablog_elasticsearch` 에는 언더스코어(_)가 있는데, URI 표준(RFC 3986)에서 언더스코어는 호스트명에 허용되지 않는다. 그래서 Logstash가 `http://mablog_elasticsearch:9200` 으로 접속하면 파싱 오류가 난다. `elasticsearch` 라는 별명(alias)을 붙여서 해결했다.

depends_on

서비스 시작 순서를 정한다. `mablog_logstash` 는 `mablog_kafka` 와 `mablog_elasticsearch` 가 먼저 시작된 후에 시작된다. 단, "실행 시작"을 기다리는 것이지 "완전히 준비됨"을 보장하지는 않는다.

docker.sock 마운트

```
volumes:
  - /var/run/docker.sock:/var/run/docker.sock
```

Docker 소켓 파일. 이것 컨테이너에 마운트하면, 컨테이너 안에서 호스트의 **Docker**를 제어할 수 있다. Next.js 컨테이너가 `docker ps` 를 실행해서 실행 중인 컨테이너 목록을 가져오는 데 쓴다.

KAFKA_HEAP_OPTS / ES_JAVA_OPTS

```
KAFKA_HEAP_OPTS=-Xms256m -Xmx256m # JVM 최소/최대 힙 256MB
ES_JAVA_OPTS=-Xms512m -Xmx512m # Elasticsearch JVM 힙 512MB
```

Kafka, Elasticsearch, Logstash는 모두 **Java(JVM)** 기반이다. 메모리를 제한하지 않으면 OCI 무료 서버(24GB RAM 있지만) 에서 과도하게 쓸 수 있어 제한을 둔다. **-Xms** 는 초기 힙 크기, **-Xmx** 는 최대 힙 크기.

3-3. Kafka KRaft 모드

기존 Kafka는 메타데이터를 관리하기 위해 **ZooKeeper**라는 별도 서비스가 필요했다. KRaft 모드는 ZooKeeper 없이 Kafka 자체가 메타데이터를 관리한다 (Raft 합의 알고리즘 사용).

```
KAFKA_PROCESS_ROLES=broker,controller # 브로커 + 컨트롤러 역할 동시 수행
KAFKA_LISTENERS=PLAINTEXT://:9092,CONTROLLER://:9093
# 9092: 메시지 produce/consume 포트
# 9093: 클러스터 내부 컨트롤러 통신 포트
```

3-4. 유용한 Docker 명령어

명령어	설명
<code>docker-compose up -d</code>	전체 서비스 백그라운드 실행 (설정 변경 시 재적용)
<code>docker-compose up -d --build 서비스명</code>	특정 서비스만 이미지 다시 빌드하고 재시작
<code>docker-compose ps</code>	실행 중인 서비스 상태 확인
<code>docker-compose logs -f 서비스명</code>	특정 서비스 로그 실시간 확인
<code>docker-compose restart 서비스명</code>	재시작 (단, 새 볼륨/환경변수 반영 안 됨)
<code>docker ps --format '{{json .}}'</code>	실행 중인 컨테이너를 JSON 형식으로 출력
<code>docker exec -it 컨테이너명 bash</code>	실행 중인 컨테이너 안으로 접속
<code>sudo chown -R 1000:1000 ./elasticsearch_data</code>	Elasticsearch 데이터 폴더 권한 수정 (UID 1000 = ES 사용자)

4. 데이터베이스 — MongoDB

4-1. MongoDB란?

관계형 DB(MySQL, PostgreSQL)와 달리 문서(**Document**) 기반 **NoSQL**. 데이터를 JSON처럼 생긴 BSON 형식으로 저장한다. 테이블 대신 **컬렉션(Collection)**, 행 대신 **문서(Document)**.

RDB vs MongoDB 비교

RDB: Database → Table → Row → Column

MongoDB: Database → Collection → Document → Field

4-2. ConnectMongoDB.ts — 싱글톤 연결

```
let cached = (globalThis as CustomGlobalThis).mongooseCache;
if (!cached) cached = (globalThis as CustomGlobalThis).mongooseCache
= {};

export const ConnectMongoDB = async () => {
  if (cached.conn && cached.conn.connection.readyState === 1) return
  cached.conn; // 이미 연결됨
  if (cached.promise) return cached.promise; // 연결 중

  cached.promise = mongoose.connect(MONGO_URI, opts); // 새 연결
  cached.conn = await cached.promise;
  return cached.conn;
};
```

왜 싱글톤인가? Next.js는 서버리스 환경에서 API 요청마다 코드가 실행된다. 매번 새 DB 연결을 만들면 연결이 수백 개가 생긴다. `globalThis` (전역 객체)에 연결을 캐싱해서 재사용한다.

`readyState === 1` 이면 이미 연결된 상태다.

중요 옵션: `authSource: 'admin'` — MongoDB 인증을 'admin' 데이터베이스에서 확인하도록 지정. docker-compose.yml에서 MONGO_INITDB_ROOT_USERNAME/PASSWORD로 설정한 계정이 admin DB에 만들어지기 때문.

4-3. 스키마 정의 (Mongoose)

Mongoose는 MongoDB를 Node.js에서 편리하게 쓰는 ODM(Object Document Mapper) 라이브러리. 스키마로 데이터 구조와 제약을 정의한다.

Posts 스키마

```
const postsSchema = new Schema({
  title:    { type: String, required: true, trim: true }, // 필수, 앞뒤 공백 제거
  content:  { type: String, required: true },             // 마크다운 원문
  category: { type: String, required: true },
  date:     { type: Date, default: Date.now },           // 없으면 현재 시각
  slug:     { type: String, required: true, unique: true }, // 중복 불가 (URL용)
}, { versionKey: false }); // __v 필드 제거
```

slug: URL에 쓰이는 식별자. 예를 들어 포스트 URL이 `/post/elk-pipeline-study` 면 slug는 `elk-pipeline-study` 다. `unique: true`로 중복 방지.

TypeScript 타입 계층

```
BaseInsertPostType           // title, content, category, date, slug
├─ IPostData                 // + _id (MongoDB ObjectId)
│   └─ IPostDataWithHtml     // + contentHtml (마크다운 → HTML 변환본)
```

MongoDB Aggregation (카테고리 통계)

```
await ModelPostsSetting.aggregate([
  { $group: { _id: '$category', count: { $sum: 1 } } }, // 카테고리별 그룹
  { $sort: { count: -1 } },                             // 많은 순 정렬
]);
```

SQL의 `GROUP BY category ORDER BY count DESC` 와 동일. Aggregation Pipeline은 데이터를 단계별로 처리한다. CATEGORY_FILTER 위젯에서 사용.

4-4. 쿼리 패턴

코드

SQL 등가

설명

<code>find(filter).sort({date:-1}).skip(n).limit(m)</code>	SELECT ... ORDER BY date DESC LIMIT m OFFSET n	페이지네이 션
<code>\$regex: query, \$options: 'i'</code>	LIKE '%query%' (대 소문자 무시)	검색
<code>\$or: [{title:...}, {category:...}]</code>	WHERE title LIKE ... OR category LIKE ...	다중 필드 검 색
<code>findByIdAndDelete(id)</code>	DELETE WHERE id=...	ID로 삭제
<code>findOneAndUpdate({}, data, {upsert:true})</code>	INSERT OR UPDATE	없으면 생성, 있으면 수정
<code>.lean()</code>	-	순수 JS 객체 반환 (Mongoose 메서드 제거, 성능 향상)

5. API 설계 — Next.js App Router

5-1. API 엔드포인트 목록

메서드	URL	인증	설명
GET	/api/controller/GET/posts	없음	포스트 목록 (페이지네이션, 검색)
POST	/api/controller/INSERT/ posts	관리자 or API키	포스트 추가
POST	/api/controller/UPDATE	관리자	포스트 수정
DELETE	/api/controller/DELETE/ posts	관리자	포스트 삭제
GET	/api/controller/GET/stack	없음	기술스택 목록
POST	/api/controller/INSERT/ stack	관리자	스택 추가
DELETE	/api/controller/DELETE/ stack	관리자	스택 삭제
GET	/api/controller/GET/ categories	없음	카테고리별 포스트 수
GET	/api/system/server	없음	OCI 서버 CPU/메모리/디스크/업타임
GET	/api/system/docker	없음	Docker 컨테이너 목록 (SSE 스트림)
GET	/api/system/github	없음	GitHub 최신 커밋 목록
GET	/api/system/raspberry	없음	Raspberry Pi 상태 조회
POST	/api/system/raspberry	API키	Raspberry Pi 상태 저장
GET	/api/system/nginx	없음	nginx 로그 최신 20건 (Elasticsearch)

5-2. apiClient.ts — 클라이언트 fetch 래퍼

각 API를 직접 fetch로 호출하면 URL, 헤더, 에러 처리를 매번 반복해야 한다. `apiClient.ts` 에 모아서 함수 하나로 호출한다.

```
const jsonPost = (body: unknown): RequestInit => ({
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(body),
});

export const getPosts = async (page = 1, limit = 10, q = '') => {
  const params = new URLSearchParams({ page: String(page), limit:
String(limit) });
  if (q) params.set('q', q);
  const res = await fetch(`/api/controller/GET/posts?${params}`);
  if (!res.ok) throw new Error('getPosts 실패');
  return res.json();
};
```

컴포넌트에서는 `const { posts, total } = await getPosts(1, 7, '검색어');` 처럼 간단하게 호출.

5-3. 포스트 GET — 페이지네이션 + 검색

```
// route.ts
export async function GET(request: NextRequest) {
  const { searchParams } = new URL(request.url);
  const page = Math.max(1, Number(searchParams.get('page') ?? 1));
  const limit = Math.max(1, Number(searchParams.get('limit') ?? 10));
  const query = searchParams.get('q') ?? '';
  return NextResponse.json(await GetPostsAllData(page, limit,
query));
}

// GETmDBTypePosts.ts - 실제 DB 조회
const filter = query
  ? { $or: [
    { title: { $regex: query, $options: 'i' } },
    { category: { $regex: query, $options: 'i' } },
  ] }
  : {};

const posts = await ModelPostsSetting.find(filter)
  .sort({ date: -1 })
  .skip((page - 1) * limit)
```

```
.limit(limit)
.lean();
```

5-4. 서버 상태 API — /proc/stat 직접 읽기

```
function ReadCPU() {
  const stat = fs.readFileSync('/proc/stat', 'utf8');
  const line = stat.split('\n')[0]; // "cpu 3357 0 4313 1234567 ..."
  const fields = line.split(/\s+/).slice(1).map(Number);
  const idle = fields[3]; // 4번째 값이 idle 시간
  const total = fields.reduce((a, b) => a + b, 0); // 전체 합
  return { idle, total };
}
```

`/proc/stat` 는 리눅스 커널이 제공하는 가상 파일. CPU 사용 시간을 클럭 틱(clock tick) 단위로 기록한다. 두 번 읽어서 **차이(delta)**를 계산하면 그 사이의 CPU 사용률이 나온다.

```
CPU 사용률 = (1 - idle delta / total delta) × 100
```

5-5. Docker API — SSE (Server-Sent Events)

일반 REST API는 클라이언트가 요청해야 데이터를 받는다. SSE는 **서버가 연결을 유지하면서 계속 데이터를 push**한다. Docker 컨테이너 상태처럼 실시간 업데이트에 적합하다.

```
// 서버 (route.ts)
const stream = new ReadableStream({
  async start(controller) {
    const send = async () => {
      const containers = await getContainers();
      controller.enqueue(encoder.encode(`data: ${
        JSON.stringify(containers)}\n\n`));
      // SSE 형식: "data: JSON\n\n" - 빈 줄 두 개가 메시지 구분자
    };
    await send(); // 연결 즉시 첫 전송
    const interval = setInterval(send, 5000); // 5초마다 반복
    request.signal.addEventListener('abort', () => {
      clearInterval(interval); controller.close(); // 클라이언트 끊으면 정
    리
  });
},
});
return new Response(stream, {
  headers: { 'Content-Type': 'text/event-stream', 'Cache-Control':
    'no-cache' },
});
```

```
});  
  
// 클라이언트 (getDockerState.tsx)  
const es = new EventSource('/api/system/docker');  
es.onmessage = (e) => {  
  setContainers(JSON.parse(e.data));  
};  
return () => es.close(); // 컴포넌트 언마운트 시 연결 종료
```


6. 인증 — NextAuth + GitHub OAuth

6-1. OAuth 흐름

```

사용자 "GitHub 로그인" 클릭
|
▼
Next.js → GitHub로 리다이렉트
(https://github.com/login/oauth/authorize?client_id=...)
|
▼
사용자가 GitHub에서 권한 승인
|
▼
GitHub → 우리 서버로 콜백 (code 파라미터 포함)
(/api/auth/callback/github?code=xxx)
|
▼
NextAuth가 code → access_token 교환 (GitHub API 호출)
|
▼
access_token으로 GitHub 유저 정보 조회
(profile.id 확인)
|
▼
JWT 토큰 생성 (isAdmin 여부 포함) → 브라우저 쿠키 저장

```

6-2. 관리자 판별

```

async jwt({ token, profile, account }) {
  if (account?.provider === 'github' && profile) {
    const githubProfile = profile as { id: number };
    token.isAdmin = githubProfile.id ===
      Number(process.env.GIT_ADMIN_ID);
    // GitHub 계정 숫자 ID와 환경변수를 비교
  }
}
```

```
    return token;
}
```

GitHub 계정의 숫자 ID(`process.env.GIT_ADMIN_ID`)와 일치하면 관리자. 환경변수에 본인 GitHub ID를 저장한다. GitHub ID는 `https://api.github.com/users/유저명` 에서 `id` 필드로 확인 가능.

6-3. 타이밍 공격 방어 (API Key 비교)

```
// 일반 비교 (취약)
if (provided === apiKey) ... // 틀린 글자 위치에 따라 응답 시간이 다름

// 상수 시간 비교 (XOR)
let diff = 0;
for (let i = 0; i < apiKey.length; i++) {
    diff |= provided.charCodeAt(i) ^ apiKey.charCodeAt(i);
}
return diff === 0;
// XOR: 같으면 0, 다르면 1 → 모든 글자를 끝까지 비교 → 시간이 일정
```

타이밍 공격(Timing Attack): 응답 시간의 미세한 차이로 비밀키를 유추하는 공격. `===` 로 비교하면 앞 글자가 맞을수록 더 오래 걸린다. XOR 비교는 항상 같은 시간이 걸려 공격을 방어한다.

6-4. 이중 인증 (포스트 INSERT)

```
function isAuthorized(request, isAdmin): boolean {
    if (isAdmin) return true; // GitHub 관리자 로그인 → OK

    // 또는 API Key로도 허용 (GitHub Actions 자동 포스팅용)
    const authHeader = request.headers.get('authorization');
    const provided = authHeader?.startsWith('Bearer ') ?
    authHeader.slice(7) : '';
    // ... 상수 시간 비교 ...
}
```

포스트 작성은 두 가지 방법으로 가능하다: ① 관리자가 직접 로그인해서 작성, ② GitHub Actions 같은 자동화 스크립트가 API Key를 Header에 포함해서 POST 요청.


```
RT=$request_time';  
access_log /var/log/nginx/access.log custom_log;
```

실제 로그 한 줄 예시:

```
172.18.0.1 - - [05/Apr/2026:23:14:29 +0000] "GET /favicon.ico HTTP/  
2.0" 200 2238 "-" "Mozilla/5.0 ..." RT=0.011
```

7-3. Logstash Grok 파싱

Grok은 정규식에 이름을 붙인 패턴이다. 텍스트 한 줄을 여러 필드로 분해한다.

```
grok {  
  match => {  
    "message" => "%{IPORHOST:client_ip} - %{DATA:user} \[%  
{HTTPDATE:log_timestamp}\] \"%{WORD:method} %{DATA:path} HTTP/%  
{NUMBER:http_version}\" %{NUMBER:status:int} %{NUMBER:bytes:int} \"%  
{DATA:referrer}\" \"%{DATA:user_agent}\" RT=%  
{NUMBER:response_time:float}"  
  }  
}
```

패턴 설명:

패턴	의미	결과 필드
%{IPORHOST:client_ip}	IP 또는 호스 트명	client_ip: "172.18.0.1"
%{HTTPDATE:log_timestamp}	HTTP 날짜 형식	log_timestamp: "05/Apr/ 2026:23:14:29 +0000"
%{WORD:method}	공백 없는 단 어	method: "GET"
%{NUMBER:status:int}	숫자 → 정수 형	status: 200
% {NUMBER:response_time:float}	숫자 → 실수 형	response_time: 0.011

7-4. Elasticsearch 인덱스

Elasticsearch에서 **인덱스(Index)**는 RDB의 테이블과 비슷하다. 날짜별로 인덱스를 나눈다
(`nginx-logs-2026.04.06`). `nginx-logs-*` 로 와일드카드 검색 가능.

```
// Next.js에서 ES 검색
POST http://elasticsearch:9200/nginx-logs-*/_search
{
  "size": 20,
  "sort": [{ "@timestamp": { "order": "desc" } }],
  "_source": ["client_ip", "method", "path", "status", ...]
}
// _source: 가져올 필드만 지정 (전체 문서 대신)
// sort: 최신순 정렬
```

7-5. Apache Kafka란?

Kafka는 **분산 메시지 큐(Message Queue)**다. 데이터를 **토픽(Topic)**에 쌓으면, 여러 소비자(Consumer)가 나중에 가져갈 수 있다. 실시간 데이터 스트리밍에 쓴다.

지금 이 프로젝트에서 Kafka의 역할:

Logstash가 nginx 로그를 Elasticsearch와 Kafka 양쪽에 동시 전송한다. Kafka에는 **nginx-logs** 토픽에 쌓인다. 지금은 Kafka에서 데이터를 가져가는 Consumer가 없지만, 나중에 실시간 알림, 통계 집계 등을 붙일 수 있다.

7-6. Kibana 접근 (SSH 터널)

Kibana는 보안상 외부에 직접 노출하지 않는다. SSH 터널로 로컬 포트와 서버 포트를 연결한다.

```
ssh -L 5601:localhost:5601 -i 키파일.pem ubuntu@서버IP -N
# -L 5601:localhost:5601 : 로컬 5601 → 서버의 localhost:5601
# -N : 명령어 실행 없이 터널만 유지
# -i : 개인키 파일 지정

# 연결 후 브라우저에서: http://localhost:5601
```

8. 시스템 모니터링 위젯

8-1. OCI 서버 상태 위젯

Next.js 서버가 직접 쉘 스크립트를 실행해서 시스템 정보를 읽는다.

```
// route.ts
const { stdout } = await execAsync("sh /opt/ma_blog/read_os/status.sh");
const data = JSON.parse(stdout);
// { memory: { used: "1234", total: "24000" }, disk: { used: "20G", total: "47G" }, uptime: "112d 1h 6m" }
```

`/opt/ma_blog/read_os/status.sh` 는 호스트에 있는 쉘 스크립트인데, docker-compose.yml에서 볼륨으로 컨테이너 안에 마운트되어 있다. `free`, `df`, `/proc/uptime` 으로 정보를 수집하고 JSON으로 출력한다.

업타임 계산 (/proc/uptime)

```
UPTIME_SEC=$(awk '{print int($1)}' /proc/uptime)
# /proc/uptime 첫 번째 값 = 부팅 후 경과 초
DAYS=$((UPTIME_SEC / 86400))          # 86400 = 60*60*24
HOURS=$(( (UPTIME_SEC % 86400) / 3600 ))
MINS=$(( (UPTIME_SEC % 3600) / 60 ))
# Alpine/BusyBox에는 "uptime -p" 명령이 없어서 직접 계산
```

프론트엔드 표시 (fmtMem)

```
const fmtMem = (mb: number) =>
  mb >= 1024 ? `${(mb / 1024).toFixed(1)}G` : `${mb}M`;
// 1024MB 이상이면 G로 변환. 예) 1536MB → "1.5G"
```

8-2. Docker 컨테이너 위젯 (SSE)

5초마다 `docker ps` 를 실행해서 컨테이너 목록을 SSE로 스트리밍한다. 컨테이너 이름으로 역할을 판별해서 색상 배지를 표시한다.

```
function getRole(name: string): Role {
  if (name.includes('proxy') || name.includes('certbot')) return
  'proxy';
  if (name.includes('nextjs')) return
  'server';
  if (name.includes('mongodb')) return
  'data';
  if (name.includes('kafka') || name.includes('elastic') ||
    name.includes('logstash') || name.includes('kibana')) return
  'pipeline';
  return 'security';
}
```

8-3. Raspberry Pi 상태

Raspberry Pi 4가 주기적으로 자신의 상태(CPU, 메모리, 온도 등)를 블로그 API에 POST로 전송한다. 블로그는 MongoDB에 저장하고, 위젯이 GET으로 조회한다.

```
Raspberry Pi (홈 네트워크)
└─ 크론탭으로 N분마다 실행
    │   POST /api/system/raspberry
    │   Authorization: Bearer API_KEY
    ▼
mablog_nextjs → MongoDB (upsert - 항상 1개 문서만 유지)
    │
    ▼ GET /api/system/raspberry
raspberrystatus.tsx (위젯)
```

```
// upsert: 문서가 없으면 생성, 있으면 업데이트
await ModelRaspberry.findOneAndUpdate(
  {}, // 조건: 아무 문서나 (첫 번째)
  { ...body, updatedAt: new Date() }, // 업데이트할 내용
  { upsert: true, new: true } // 없으면 생성, 결과는 새 문서 반환
);
```

8-4. GitHub Activity 위젯

```
// 1. GitHub Events API로 PushEvent 목록
GET https://api.github.com/users/PasserbyMa/events?per_page=30
```

```
// 2. 각 PushEvent의 head SHA로 실제 커밋 메시지 조회
GET https://api.github.com/repos/{owner}/{repo}/commits/{sha}

// next: { revalidate: 300 } - 5분 동안 캐시 (GitHub API Rate Limit 방어)
```

Rate Limit: GitHub API는 인증 없이 시간당 60회 제한이 있다. `revalidate: 300` 으로 5분 캐시를 걸어서 반복 요청을 방지한다.

9. 프론트엔드 구조

9-1. Next.js App Router 핵심 개념

개념	설명
RSC (React Server Component)	서버에서 렌더링. DB 직접 접근 가능. 클라이언트로 JS 안 보냄
'use client'	이 파일은 브라우저에서 실행. useState, useEffect 사용 가능
'use server'	서버에서만 실행. DB 접근, 환경변수 사용
route.ts	API 엔드포인트 파일. export async function GET/POST/DELETE/...
page.tsx	실제 페이지 컴포넌트. 폴더 경로 = URL
layout.tsx	여러 페이지에 공통으로 감싸는 레이아웃

9-2. 모달 스택 시스템

여러 모달이 겹칠 때 (more → fix 순서로 열기) 스택 자료구조로 관리한다.

```
type ModalType = 'more' | 'recentDetail' | 'add' | 'fix';
const [modalStack, setModalStack] = useState<ModalType[]>([]);

const activeModal = modalStack[modalStack.length - 1] ?? null; // 최상
단 모달
const openModal = (type: ModalType) => setModalStack(prev =>
[...prev, type]);
const closeModal = () => setModalStack(prev => prev.slice(0, -1)); //
마지막 제거
const closeAll = () => setModalStack([]);

// 예: ['more', 'fix'] → fix가 more 위에 뜸
// fix 닫으면 → ['more'] → more로 복귀
```

9-3. 검색 Debounce

```
const [searchInput, setSearchInput] = useState(''); // 입력창 값
const [searchQuery, setSearchQuery] = useState(''); // 실제 검색에 쓰는 값

useEffect(() => {
  const timer = setTimeout(() => {
    setSearchQuery(searchInput); // 300ms 후에 반영
    setCurPage(1);
  }, 300);
  return () => clearTimeout(timer); // 새 입력이 오면 이전 타이머 취소
}, [searchInput]);
```

Debounce: 키 입력마다 API 요청하면 너무 많은 요청이 발생한다. 마지막 입력 후 300ms가 지나야 실제 검색이 실행된다.

9-4. 마크다운 → HTML 변환

포스트 content는 마크다운 원문으로 저장한다. 조회할 때 HTML로 변환해서 `contentHtml`에 담는다.

```
// GETmDBTypePosts.ts
const convert = await ConvertMarkdownToHtml(p.content);
return { ...p, contentHtml: convert.contentHtml };

// 클라이언트에서 렌더링
<div dangerouslySetInnerHTML={{ __html: post.contentHtml }} />
```

dangerouslySetInnerHTML: XSS(크로스 사이트 스크립팅) 취약점이 될 수 있다. 사용자 입력이 직접 HTML로 삽입되면 스크립트가 실행될 수 있다. 이 프로젝트에서는 관리자만 포스트를 작성하므로 신뢰할 수 있는 입력이다.

9-5. 컴포넌트 데이터 흐름

```
page.tsx (서버 컴포넌트)
| DB에서 초기 데이터 조회 (postData, stackData)
▼
mainPageComponent.tsx (클라이언트 컴포넌트)
| 상태(state): modalStack, p_data, recentPosts, ...
|
```

```
└─ AboutMe          ← stackData 전달
└─ GetServerState   ← 자체 fetch (/api/system/server)
└─ DockerContainersBox ← SSE 구독 (/api/system/docker)
└─ GithubActivity   ← 자체 fetch (/api/system/github)
└─ RaspberryStatus  ← 자체 fetch (/api/system/raspberry)
└─ NginxLogWidget    ← 자체 fetch (/api/system/nginx, 15초마다)
└─ CategoryFilter    ← 자체 fetch (/api/controller/GET/categories)
└─ ClockWidget       ← 자체 상태 (setInterval)
```

10. 보안 설계

항목	방법	이유
HTTPS	Nginx + Let's Encrypt (certbot)	데이터 암호화, SEO
MongoDB 외부 노출 차단	127.0.0.1:27017 (로컬만)	외부에서 DB 직접 접근 불가
Kibana 외부 노출 차단	127.0.0.1:5601 (SSH 터널만)	ES 데이터 보호
ES 보안 비활성화	xpack.security.enabled=false	내부망만 접근, 개발 편의
관리자 인증	GitHub OAuth + JWT	GitHub ID 일치 시만 관리자
API Key 인증	Bearer 토큰 + XOR 상수시간 비교	타이밍 공격 방어
환경변수	.env.production (git 제외)	비밀값 소스코드 노출 방지
Nginx 내부 전달	expose로 8080 노출, ports 아님	Next.js 직접 접근 차단

Nginx 리버스 프록시의 역할

인터넷 → Nginx(:80/:443) → Next.js(:8080, 내부)

```
# 외부는 Nginx만 보임
# Next.js는 Docker 내부 네트워크에만 존재
# Nginx가 HTTPS 처리, 압축, 정적파일 캐싱 등 담당
```

11. 용어 사전

용어	설명
Docker	애플리케이션을 컨테이너(격리된 환경)로 실행하는 플랫폼
Docker Compose	여러 컨테이너를 YAML 파일 하나로 정의/실행하는 도구
컨테이너 (Container)	격리된 프로세스 실행 환경. 가상머신보다 가볍고 빠름
이미지 (Image)	컨테이너의 설계도. Docker Hub에서 내려받음
볼륨 (Volume)	컨테이너 데이터를 호스트에 영구 저장하는 방법
네트워크 (Docker Network)	컨테이너 간 통신을 위한 가상 네트워크
Nginx	웹 서버 / 리버스 프록시 / 로드밸런서
리버스 프록시	클라이언트 요청을 받아 내부 서버로 전달하는 중간 서버
HTTPS / TLS	HTTP 암호화 버전. TLS 인증서가 필요
Let's Encrypt	무료 HTTPS 인증서 발급 기관. certbot으로 자동 갱신
MongoDB	문서 기반 NoSQL 데이터베이스
Mongoose	Node.js에서 MongoDB를 편리하게 쓰는 ODM 라이브러리
스키마 (Schema)	데이터 구조와 유효성 검사 규칙 정의
REST API	HTTP 메서드(GET/POST/PUT/DELETE)로 자원을 다루는 API 설계 방식
JWT (JSON Web Token)	로그인 정보를 담은 암호화된 토큰. 쿠키에 저장
OAuth	제3자(GitHub) 인증을 위임하는 프로토콜
NextAuth	Next.js용 인증 라이브러리
SSE (Server-Sent Events)	서버가 클라이언트에 단방향으로 이벤트를 push하는 기술
WebSocket	양방향 실시간 통신. SSE와 달리 클라이언트도 서버로 보낼 수 있음
RSC (React Server Component)	서버에서 렌더링되는 React 컴포넌트. JS를 클라이언트에 안 보냄

Elasticsearch	분산 검색/분석 엔진. JSON 문서 저장 및 빠른 검색
Logstash	데이터 수집/변환/전송 파이프라인 도구
Kibana	Elasticsearch 데이터 시각화 웹 UI
ELK Stack	Elasticsearch + Logstash + Kibana 조합
Apache Kafka	분산 메시지 큐. 대용량 실시간 데이터 스트리밍
Topic (Kafka)	메시지를 분류하는 채널. 폴더 같은 개념
KRaft	ZooKeeper 없이 Kafka 자체로 메타데이터를 관리하는 모드
Grok	Logstash에서 텍스트를 구조화된 필드로 파싱하는 패턴 언어
/proc/stat	리눅스 커널이 제공하는 CPU 사용 통계 가상 파일
/proc/uptime	리눅스 부팅 후 경과 시간을 초 단위로 제공하는 가상 파일
OCI	Oracle Cloud Infrastructure. 무료 ARM 서버 제공
JVM 힙 (Heap)	Java 프로그램이 객체를 저장하는 메모리 영역
Xms / Xmx	JVM 초기/최대 힙 크기 설정 옵션
타이밍 공격	응답 시간 차이로 비밀 정보를 유추하는 사이버 공격
XOR	비트 연산. 같으면 0, 다르면 1. 상수 시간 비교에 활용
Rate Limit	API 호출 횟수 제한. GitHub API는 비인증 시 시간당 60회
Debounce	연속 이벤트 중 마지막 것만 처리. 검색창 입력에 활용
upsert	데이터가 없으면 INSERT, 있으면 UPDATE (update + insert 합성어)
lean()	Mongoose 메서드. 순수 JS 객체 반환으로 성능 향상
sincedb	Logstash file input이 어디까지 읽었는지 기록하는 파일
SSH 터널	SSH 연결로 원격 포트를 로컬에서 사용 가능하게 하는 기술
Slug	URL에 사용되는 사람이 읽기 쉬운 식별자. 예: "elk-pipeline-study"
App Router	Next.js 13+ 라우팅 방식. 파일 경로 = URL 경로

12. 학습 로드맵

현재 수준 진단

영역	현재 수준	근거
Next.js / React	★★★★☆ 중급	App Router, RSC, SSE, 실무 경험
TypeScript	★★★★☆ 중급	타입 정의, 인터페이스 활용
MongoDB / Mongoose	★★☆☆☆ 초중급	CRUD, Aggregation, 스키마 설계
Docker	★★☆☆☆ 초중급	Compose로 다중 서비스 운영 경험
Nginx	★★☆☆☆ 초중급	리버스 프록시, HTTPS 설정
Linux	★★☆☆☆ 초중급	/proc 파일, 셸 스크립트, 권한
Kafka	★☆☆☆☆ 입문	구축 경험 있으나 개념 학습 필요
Elasticsearch	★☆☆☆☆ 입문	기본 검색 쿼리, 인덱스 개념
보안	★★☆☆☆ 초중급	타이밍 공격 방어, OAuth 구현

단기 학습 목표 (1-2개월)

- MongoDB Aggregation 심화**
 - \$lookup (JOIN), \$unwind, \$project
 - 실습: 카테고리별 월간 포스트 수 집계
- Docker 심화**
 - Dockerfile 최적화 (멀티 스테이지 빌드)
 - Docker 네트워크 종류 (bridge, host, overlay)
 - 헬스체크(healthcheck) 설정
- Nginx 심화**
 - upstream, load_balancing
 - rate limiting (요청 속도 제한)
 - gzip 압축 설정

중기 학습 목표 (3-6개월)

1. Kafka 개념 심화

- Consumer Group, Partition, Offset 개념
- nginx 로그 Kafka Consumer 만들기 (실시간 통계)

2. Elasticsearch 심화

- 매핑(Mapping), 분석기(Analyzer)
- Kibana Dashboard 만들기 (nginx 로그 시각화)

3. CI/CD

- GitHub Actions 워크플로우 작성
- 자동 테스트 + 자동 배포 파이프라인

이 프로젝트에서 바로 연습할 수 있는 것들

- Kibana에서 nginx 로그 대시보드 만들기 (파이 차트, 시계열 그래프)
- Logstash sincedb 경로 바꿔서 재시작 시 로그를 반복 읽지 않게 하기
- MongoDB에 포스트 조회수(views) 필드 추가해서 인기 포스트 기능 만들기
- Nginx rate limiting 추가해서 DDoS 방어
- Raspberry Pi 온도 데이터를 시계열로 Elasticsearch에 저장하기
- GitHub Actions로 main 브랜치 push 시 자동 docker-compose up 배포

참고 자료

주제	자료
Next.js App Router	nextjs.org/docs (공식 문서)
MongoDB	mongodb.com/docs/manual
Docker	docs.docker.com
Elasticsearch	elastic.co/guide/en/elasticsearch/reference
Kafka	kafka.apache.org/documentation
Logstash Grok 패턴	grokdebugger.com (온라인 테스트)
Nginx	nginx.org/en/docs